

PRAKTIKUM SISTEM TERTANAM

JOBSHEET 02 DASAR-DASAR BAHASA VHDL DAN ALUR SIMULASI VIVADO



Disusun oleh :

DENTA HERO PRADANA

25090620023

C2

**PROGRAM STUDI D4 TEKNIK ELEKTRONIKA
DEPARTEMEN TEKNIK ELEKTRO DAN ELEKTRONIKA
FAKULTAS VOKASI
UNIVERSITAS NEGERI YOGYAKARTA**

**SEMESTER GASAL
2026/2027**

I. TUJUAN PRAKTIKUM

1. Mahasiswa mampu menjelaskan struktur dasar kode VHDL: library, entity, dan architecture.
2. Mahasiswa mampu menjelaskan dan menggunakan tipe data std_logic, std_logic_vector, signed, dan unsigned.
3. Mahasiswa mampu membedakan pernyataan konkuren (concurrent statement) dan pernyataan sekuensial di dalam process.
4. Mahasiswa mampu menulis testbench VHDL sederhana dan menjalankan simulasi perilaku (behavioral simulation) menggunakan simulator XSIM bawaan Vivado.
5. Mahasiswa mampu membaca dan menginterpretasikan bentuk gelombang (waveform) hasil simulasi.

II. DASAR TEORI

a. Struktur Dasar Kode VHDL

Setiap rancangan VHDL minimal terdiri atas tiga bagian: (1) deklarasi library, memuat pustaka standar yang dipakai; (2) entity, mendeskripsikan antarmuka (daftar port input/output) modul; dan (3) architecture, mendeskripsikan perilaku/isi modul tersebut. Satu entity dapat memiliki lebih dari satu architecture (misalnya untuk perbandingan gaya perancangan), meskipun pada praktikum ini setiap entity cukup memiliki satu architecture.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity nama_entity is
    Port (
        a : in STD_LOGIC;
        b : in STD_LOGIC;
        y : out STD_LOGIC
    );
end nama_entity;

architecture Behavioral of nama_entity is
    -- deklarasi signal internal (opsional)
    diletakkan di sini
begin
    -- pernyataan konkuren / process diletakkan
    di sini
```

```

        y <= a and b;
end Behavioral;

```

Syntax 2.1 Kerangka Umum Kode VHDL

b. Tipe Data pada VHDL

VHDL standar (IEEE 1076) menyediakan tipe bit yang hanya bernilai '0' atau '1'. Namun, praktik industri hampir selalu menggunakan tipe `'std_logic'` dari pustaka IEEE.STD_LOGIC_1164, yang mendukung sembilan nilai logika (di antaranya: '0', '1', 'Z' (high-impedance), 'X' (tidak diketahui/konflik), 'U' (belum diinisialisasi), '-' (don't care)) sehingga jauh lebih realistis untuk simulasi maupun sintesis perangkat keras nyata. Untuk bus/vektor bit, digunakan `'std_logic_vector'`. Untuk operasi aritmatika (penjumlahan, pengurangan, perbandingan) pada bus, digunakan tipe `'unsigned'` dan `'signed'` dari pustaka IEEE.NUMERIC_STD, karena `std_logic_vector` murni tidak mendefinisikan operator aritmatika. Konversi antar tipe dilakukan eksplisit, misalnya `to_integer(unsigned(sinyal))` atau `std_logic_vector(unsigned_signal)`.

Tipe Data	Pustaka	Kegunaan Utama
<code>std_logic</code>	IEEE.STD_LOGIC_1164	Sinyal 1-bit (9-value logic)
<code>std_logic_vector</code>	IEEE.STD_LOGIC_1164	Bus/vektor bit tanpa makna aritmatika
<code>unsigned</code>	IEEE.NUMERIC_STD	Bus dengan operasi aritmatika tak bertanda
<code>signed</code>	IEEE.NUMERIC_STD	Bus dengan operasi aritmatika bertanda (2's complement)
<code>integer</code>	bawaan bahasa	Konstanta/indeks, umumnya untuk generic & loop

Tabel 2.1 Ringkasan tipe data VHDL yang umum dipakai

c. Pernyataan Konkuren vs Process (Sekuensial)

Di luar process, seluruh pernyataan pada arsitektur VHDL bersifat konkuren (paralel) — urutan penulisan tidak memengaruhi hasil, karena masing-masing mendeskripsikan potongan perangkat keras yang berjalan serentak. Contohnya adalah signal assignment langsung (`y <= a and b;`), `whenelse`, dan `with-select-when`. Sebaliknya, di dalam blok `'process(...)'`, pernyataan dieksekusi secara sekuensial dari atas ke bawah (mirip perangkat lunak), namun tetap mendeskripsikan perangkat keras kombinasional atau sekuensial tergantung sensitivity list dan isinya.

```

-- Contoh with-select (konkuren): multiplexer 4-ke-
1
    with sel select
        y <= in0 when "00",
            in1 when "01",
            in2 when "10",
            in3 when others;
-- Contoh process (sekuensial di dalamnya, tetap
-- kombinasional keseluruhan
-- karena SEMUA sinyal input ada di
-- sensitivity list)
    process(sel, in0, in1, in2, in3)
    begin
        case sel is
            when "00" => y <= in0;
            when "01" => y <= in1;
            when "10" => y <= in2;
            when others => y <= in3;
        end case;
    end process;

```

Kode 2.2 Dua gaya penulisan multiplexer: konkuren vs process

d. Testbench VHDL

Testbench adalah rancangan VHDL khusus untuk verifikasi, tidak untuk disintesis ke perangkat keras. Entity testbench tidak memiliki port (kosong), karena testbench berperan sebagai "dunia luar" yang membangkitkan stimulus bagi Design Under Test (DUT) dan mengamati responsnya. Pewaktuan stimulus umumnya diatur menggunakan pernyataan wait for <waktu>; di dalam sebuah process tanpa sensitivity list.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_and_gate is
end tb_and_gate;

architecture sim of tb_and_gate is
    signal a_tb, b_tb, y_tb : STD_LOGIC;
begin

    -- Instansiasi DUT (Device Under Test)
    DUT: entity work.nama_entity
        port map (
            a => a_tb,

```

```

        b => b_tb,
        y => y_tb
    );

    stim_proc: process
    begin
        a_tb <= '0';
        b_tb <= '0';
        wait for 20 ns;

        a_tb <= '0';
        b_tb <= '1';
        wait for 20 ns;

        a_tb <= '1';
        b_tb <= '0';
        wait for 20 ns;

        a_tb <= '1';
        b_tb <= '1';
        wait for 20 ns;

        assert (y_tb = '1')
            report "AND gate gagal: hasil
seharusnya 1"
            severity error;

        wait;
    end process;

end sim;

```

Kode 2.3 tb_and_gate.vhd — testbench sederhana dengan assert

e. Alur Simulasi pada Vivado (XSIM)

Vivado menyediakan simulator bawaan bernama XSIM. Berkas testbench ditambahkan pada kategori sumber Simulation Sources (terpisah dari Design Sources agar tidak ikut disintesis). Setelah dijalankan (Run Simulation → Run Behavioral Simulation), Vivado menampilkan jendela waveform tempat sinyal dapat ditambahkan dan diamati transisi nilainya terhadap waktu.

III. ALAT DAN BAHAN

1. 1 unit PC/laptop dengan AMD Vivado Design Suite.
2. Proyek Vivado yang telah dibuat pada Jobsheet 1 (atau proyek baru khusus simulasi, papan target tidak wajib diprogram pada jobsheet ini).

IV. PROSEDUR PRAKTIKUM

1. Buat sumber desain baru bernama mux4to1.vhd pada kategori Design Sources, dengan port: sel (2-bit), in0, in1, in2, in3 (masing-masing 4-bit std_logic_vector), dan y (4-bit output).
2. Tuliskan isi arsitektur menggunakan gaya process dengan case seperti pada Kode 2.2 (sesuaikan lebar bus menjadi 4-bit).
3. Buat sumber baru bernama tb_mux4to1.vhd, namun kali ini klik kanan pada berkas tersebut di panel Sources setelah dibuat, lalu pilih Set as Top dan pastikan berkas berada pada kategori Simulation Sources (klik kanan berkas → Move to Simulation Sources apabila belum sesuai).
4. Tuliskan testbench mengikuti kerangka Kode 2.3, sesuaikan nama entity DUT menjadi mux4to1, dan berikan minimal 4 skenario stimulus berbeda (variasi sel dari "00" hingga "11").
5. Pada Flow Navigator, klik Run Simulation → Run Behavioral Simulation.
6. Pada jendela waveform yang terbuka, tambahkan sinyal sel, in0–in3, dan y (bila belum tampil semua, drag dari panel Scope/Objects).
7. Ubah format tampilan (radix) sinyal 4-bit menjadi Unsigned Decimal atau Hexadecimal melalui klik kanan sinyal → Radix, agar lebih mudah dibaca.
8. Klik ikon Run All (▶▶) atau atur durasi simulasi (misalnya 200 ns) lalu jalankan ulang untuk memastikan seluruh skenario stimulus tereksekusi.
9. Amati apakah nilai y pada setiap interval waktu sesuai dengan in<sel> yang aktif pada saat itu; catat pada laporan.

V. HASIL PRAKTIKUM

A. Kode Program VHDL (RTL & Testbench)

1. Kode MUX 4-to-1 (mux4to1.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux4to1 is
    Port ( sel : in  STD_LOGIC_VECTOR (1
downto 0));
        in0 : in  STD_LOGIC_VECTOR (3
downto 0);
        in1 : in  STD_LOGIC_VECTOR (3
downto 0);
        in2 : in  STD_LOGIC_VECTOR (3
downto 0);
        in3 : in  STD_LOGIC_VECTOR (3
downto 0);
        y   : out STD_LOGIC_VECTOR (3
downto 0));
end mux4to1;

architecture Behavioral of mux4to1 is
begin
    process (sel, in0, in1, in2, in3)
    begin
        case sel is
            when "00"    => y <= in0;
            when "01"    => y <= in1;
            when "10"    => y <= in2;
            when others => y <= in3;
        end case;
    end process;
end Behavioral;
```

Syntax 5.1 Kode Program MUX 4 to 1

2. Kode Testbench MUX 4-to-1 (tb_mux4to1.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_mux4to1 is
end tb_mux4to1;

architecture sim of tb_mux4to1 is
```

```

        signal sel_tb : STD_LOGIC_VECTOR(1
downto 0) := "00";
        signal in0_tb : STD_LOGIC_VECTOR(3
downto 0) := "0001";
        signal in1_tb : STD_LOGIC_VECTOR(3
downto 0) := "0010";
        signal in2_tb : STD_LOGIC_VECTOR(3
downto 0) := "0100";
        signal in3_tb : STD_LOGIC_VECTOR(3
downto 0) := "1000";
        signal y_tb   : STD_LOGIC_VECTOR(3
downto 0);
begin
    DUT: entity work.mux4to1
        port map (
            sel => sel_tb, in0 => in0_tb,
in1 => in1_tb, in2 => in2_tb, in3 =>
in3_tb, y => y_tb
        );

    stim_proc: process
    begin
        sel_tb <= "00"; wait for 20 ns;
        assert (y_tb = "0001") report
"Error pada sel 00!" severity error;

        sel_tb <= "01"; wait for 20 ns;
        assert (y_tb = "0010") report
"Error pada sel 01!" severity error;

        sel_tb <= "10"; wait for 20 ns;
        assert (y_tb = "0100") report
"Error pada sel 10!" severity error;

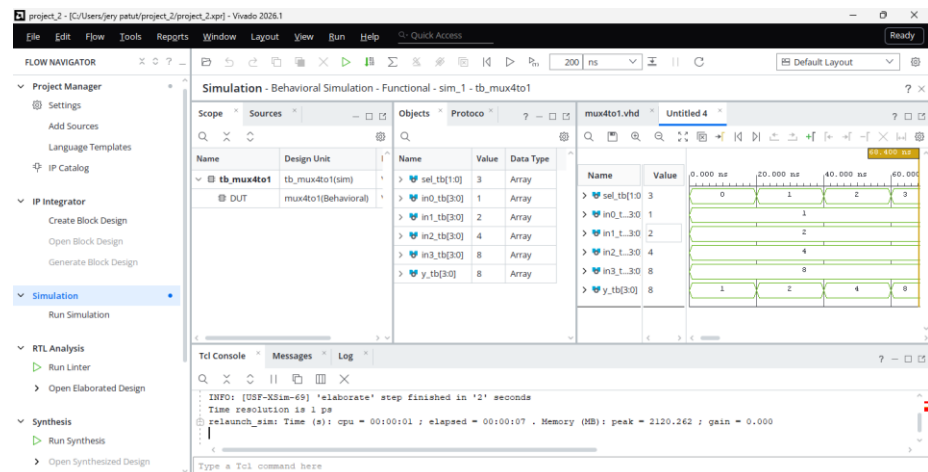
        sel_tb <= "11"; wait for 20 ns;
        assert (y_tb = "1000") report
"Error pada sel 11!" severity error;

        wait;
    end process;
end sim;

```

Syntax 5.2 Kode Testbench MUX 4-to-1

3. Hasil Eksekusi



Gambar 5.1 Hasil Eksekusi MUX 4 to 1

B. Pembahasan

Pada praktikum ini, simulasi perilaku (*behavioral simulation*) dilakukan menggunakan XSIM Vivado tanpa melalui proses sintesis ke board FPGA Basys 3. Pengujian multiplexer 4-ke-1 (mux4to1.vhd) dan 8-ke-1 (mux8to1.vhd) menunjukkan bahwa sinyal luaran *y_tb* berganti secara tepat mengikuti kombinasi sinyal pemilih *sel_tb*. Karena nilai masukan *in0* hingga *in7* diatur statis pada testbench, sinyal masukannya tampak mendatar pada *waveform*, sedangkan keluaran *y_tb* membentuk tangga perubahan sinyal yang mengonfirmasi bahwa logika kombinasional case-when berfungsi sempurna.

VI. JAWABAN TUGAS PRAKTIKUM

1. Modifikasi MUX menjadi 8-to-1 (mux8to1.vhd dan tb_mux8to1.vhd)

a. Kode Program mux8to1.vhd:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux8to1 is
    Port ( sel : in  STD_LOGIC_VECTOR (2
downto 0);
          in0, in1, in2, in3, in4, in5,
in6, in7 : in  STD_LOGIC_VECTOR (3 downto
0);
          y   : out STD_LOGIC_VECTOR (3
downto 0));
end mux8to1;

architecture Behavioral of mux8to1 is
begin
```

```

        process (sel, in0, in1, in2, in3,
in4, in5, in6, in7)
        begin
            case sel is
                when "000" => y <= in0;
                when "001" => y <= in1;
                when "010" => y <= in2;
                when "011" => y <= in3;
                when "100" => y <= in4;
                when "101" => y <= in5;
                when "110" => y <= in6;
                when others => y <= in7;
            end case;
        end process;
end Behavioral;

```

Syntax 6.1 Kode Program MUX 8 to 1

b. Kode Testbench tb_mux8to1.vhd:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_mux8to1 is
end tb_mux8to1;

architecture sim of tb_mux8to1 is
    signal sel_tb : STD_LOGIC_VECTOR(2
downto 0) := "000";
    signal in0_tb : STD_LOGIC_VECTOR(3
downto 0) := "0001";
    signal in1_tb : STD_LOGIC_VECTOR(3
downto 0) := "0010";
    signal in2_tb : STD_LOGIC_VECTOR(3
downto 0) := "0011";
    signal in3_tb : STD_LOGIC_VECTOR(3
downto 0) := "0100";
    signal in4_tb : STD_LOGIC_VECTOR(3
downto 0) := "0101";
    signal in5_tb : STD_LOGIC_VECTOR(3
downto 0) := "0110";
    signal in6_tb : STD_LOGIC_VECTOR(3
downto 0) := "0111";
    signal in7_tb : STD_LOGIC_VECTOR(3
downto 0) := "1000";

```

```

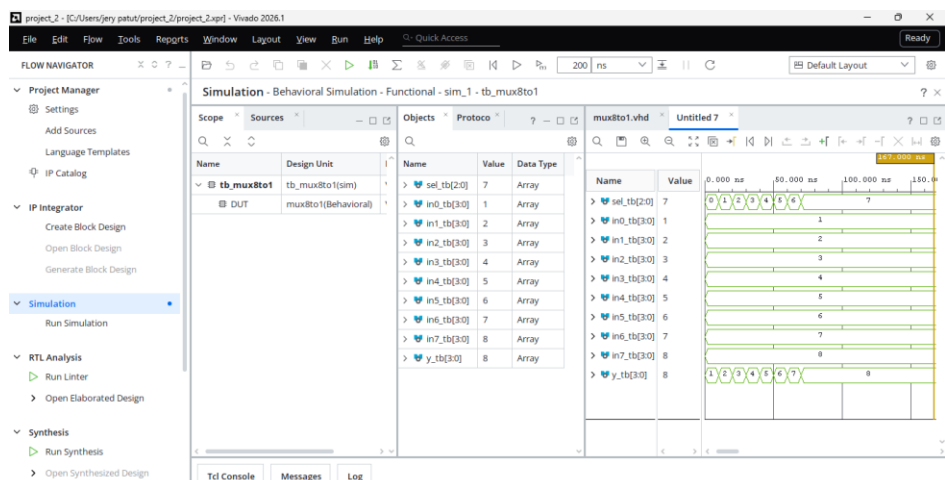
        signal y_tb    : STD_LOGIC_VECTOR(3
downto 0);
begin
    DUT: entity work.mux8to1
        port map (
            sel => sel_tb,
            in0 => in0_tb, in1 => in1_tb,
in2 => in2_tb, in3 => in3_tb,
            in4 => in4_tb, in5 => in5_tb,
in6 => in6_tb, in7 => in7_tb,
            y    => y_tb
        );

    stim_proc: process
begin
        sel_tb <= "000"; wait for 10 ns;
        sel_tb <= "001"; wait for 10 ns;
        sel_tb <= "010"; wait for 10 ns;
        sel_tb <= "011"; wait for 10 ns;
        sel_tb <= "100"; wait for 10 ns;
        sel_tb <= "101"; wait for 10 ns;
        sel_tb <= "110"; wait for 10 ns;
        sel_tb <= "111"; wait for 10 ns;
        wait;
    end process;
end sim;

```

Syntax 6.2 Kode Testbench MUX 8 to 1

c. Hasil Eksekusi MUX 8 to 1



Gambar 6.1 Hasil Eksekusi MUX 8 to 1

2. Penjelasan Fungsi Pernyataan assert pada Self-Checking Testbench

Pernyataan assert digunakan untuk menguji kebenaran sinyal luaran secara otomatis (self checking). Simulator XSIM akan mengecek apakah keluaran y_tb bernilai sesuai ekspektasi. Jika terdapat kesalahan logika, simulator akan memunculkan laporan kesalahan (error report) di Tcl Console Vivado. Hal ini menghilangkan kebutuhan untuk mengecek gelombang waveform secara manual.

3. Perbedaan signal dan variable pada VHDL

Dalam perancangan hardware menggunakan VHDL, perbedaan mendasar antara signal dan variable di dalam sebuah blok process terletak pada waktu pembaruan nilainya serta operator assignment yang digunakan:

a. Waktu Adanya Nilai Baru (Timing Update)

- variable: Nilainya diperbarui secara langsung saat baris kode tersebut dieksekusi. Artinya, jika ada baris perintah tepat di bawahnya yang membaca variabel tersebut, nilai yang terbaca adalah nilai terbaru yang baru saja diubah.
- signal: Nilainya tidak langsung berubah saat baris kodenya dibaca. VHDL menunda pembaruan sinyal (*scheduled update*) sampai seluruh instruksi di dalam blok process selesai dijalankan pada langkah simulasi tersebut. Jadi, jika kita membaca kembali sinyal tersebut di baris berikutnya dalam process yang sama, nilai yang terbaca masih nilai lama sebelum diubah.

b. Penggunaan Operator Assignment (:= vs <=)

- variable menggunakan operator assignment :=. Penggunaan simbol ini mirip seperti assignment pada bahasa pemetaan software umum (seperti C atau Pascal) karena sifat perubahannya yang seketika.
- signal menggunakan operator assignment <=. Simbol panah ini menggambarkan alur transfer data antar jalur kabel/koneksi pada perangkat keras.

Intinya Adalah variable bekerja mirip variabel software tempat nilai langsung ter-update di baris berikutnya, sedangkan signal mewakili kabel fisik tempat perubahan nilai baru benar-benar aktif setelah siklus evaluasi proses tersebut tuntas.

VII. ANALISIS & PEMBAHASAN

Berdasarkan simulasi perilaku (*behavioral simulation*) yang telah dilakukan pada modul MUX 4-to-1 dan MUX 8-to-1 menggunakan XSIM Vivado, terdapat beberapa poin penting yang dapat dianalisis:

1. Analisis Karakteristik Gelombang (Waveform)

Pada grafik hasil simulasi, sinyal masukan data (in0 hingga in7) terlihat berupa garis lurus mendatar karena nilainya diset konstan di dalam kode testbench. Sementara itu, sinyal pemilih (sel_tb) dan sinyal luaran (y_tb) membentuk pola tangga. Hal ini menganalisis bahwa luaran y_tb tidak merespon nilai masukan secara acak, melainkan secara dinamis memperbarui nilainya mengikuti jalur masukan yang ditunjuk oleh sel_tb setiap interval waktu 10–20 ns.

2. Analisis Efektivitas Self-Checking Testbench dengan assert

Penerapan fitur assert di dalam *testbench* memberikan mekanisme verifikasi otomatis. Selama proses simulasi berlangsung, Tcl Console Vivado tidak mengeluarkan notifikasi error, yang mengindikasikan bahwa seluruh nilai luaran y_tb secara presisi sama dengan nilai masukan yang diharapkan. Pendekatan ini terbukti jauh lebih efisien dan meminimalisir kesalahan manusia (*human error*) dibandingkan evaluasi manual grafik waveform.

3. Analisis Perilaku Sekuensial case-when dalam Process

Logika multiplexer diimplementasikan menggunakan blok proses dengan pernyataan case-when. Penggunaan process memastikan evaluasi dilakukan secara sekuensial saat terjadi perubahan pada sinyal-sinyal yang ada di dalam *sensitivity list* (sel, in0, in1, ...).

VIII. KESIMPULAN

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa:

1. Struktur dasar VHDL yang terdiri dari *library*, *entity* (antarmuka port), dan *architecture* (deskripsi logika) berhasil diimplementasikan untuk merancang modul kombinasional MUX 4-to-1 dan MUX 8-to-1.
2. Pengujian fungsi logika multiplexer menggunakan simulator XSIM Vivado menunjukkan keberhasilan 100%, di mana sinyal luaran y terbukti mengalirkan data masukan yang sesuai dengan kombinasi bit pada sinyal pemilih sel.
3. Fitur assert pada testbench terbukti mempermudah verifikasi desain melalui pelaporan kesalahan otomatis tanpa bergantung penuh pada penafsiran manual bentuk gelombang.
4. Perbedaan mendasar antara signal dan variable terletak pada timing pembaruan nilai (ditunda vs instan) serta operator assignment yang digunakan ($\leq$ vs $:=$).

IX. LAMPIRAN LINK REPOSITORY GITHUB

<https://github.com/dentapradana-25090620023/praktikum-sistem-tertanam--25090620023-.->